Insert this in application.properties file

#Setup JDBC connection
#make sure you have created a database in Workbench or any SQL db engine. The #database name I created was db_pmss
spring.datasource.url=jdbc:mysql://localhost:3306/**db_pmss**

#Setup db login credentials
spring.datasource.username=**root**
spring.datasource.password=**Admin.1234**

#Auto create tables based on entities
spring.jpa.database-platform=org.hibernate.dialect.MySQL5InnoDBDialect
spring.jpa.hibernate.ddl-auto=update

spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5InnoDBDialect

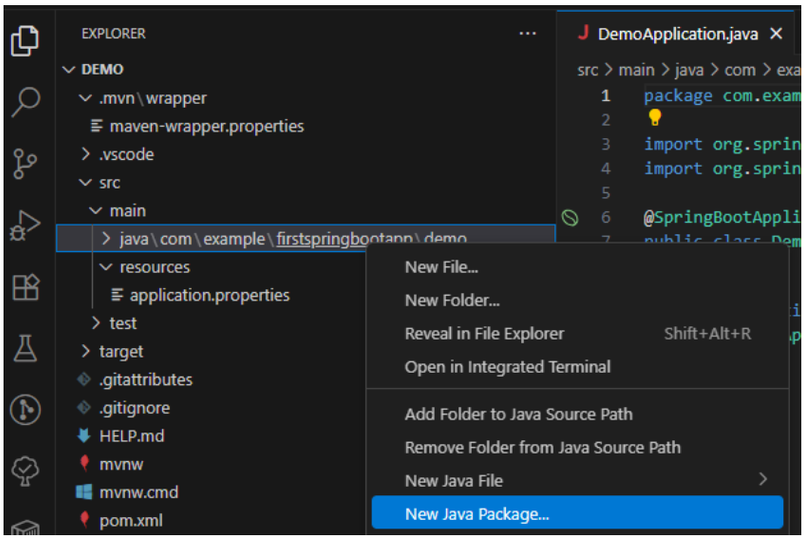
#for stack trace error not to appear
server.error.include-stacktrace=never

server.port = 8080

security.basic.enabled=false

****NOTE: Create a database named dbCRUD1**

1.) Create the following packages in this directory:



Packages to create:

entity
repository
service
controller

2.) In the **entity** package, create a class named ItemEntity with the following lines of code:

```
@Entity
@Table(name="item")
public class ItemEntity {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int itemid;
    private String name;
    private String unit;
    private float price;

    //generate constructor
    //generate setter methods for certain attributes
    //generate getter methods for all attributes
}
```

3.) In the **repository** package, create an interface named ItemRepository with the following lines of code:

```
@Repository
public interface ItemRepository extends JpaRepository<ItemEntity, Integer>{
}
```

4.) In the **service** package, create a class named ItemService with the following lines of code:

```
@Service
public class ItemService {
	@Autowired
	ItemRepository irepo;
	public ItemService() {
		// TODO Auto-generated constructor stub
	}

	//C - Create
	//method will return the student being added
	public ItemEntity postItem(ItemEntity Item) {
		return irepo.save(Item);
	}

	//R - Read
	public List<ItemEntity> getAllItems(){
		return irepo.findAll();
	}

	//U - Update
	@SuppressWarnings("finally")
	public ItemEntity putItem(int id, ItemEntity newItemDetails) {
		ItemEntity Item = new ItemEntity();

		try {
			//findById is a pre-defined method of the repository
			Item = irepo.findById(id).get();
			Item.setName(newItemDetails.getName());
			Item.setUnit(newItemDetails.getUnit());
			Item.setPrice(newItemDetails.getPrice());


		}catch(NoSuchElementException ex) {
		  throw new NameNotFoundException("Item " + id + " does not exist!");
		}finally {
			return irepo.save(Item);
		}
	}

	//D - Delete
	public String deleteItem(int id) {
		String msg = "";
		if (irepo.findById(id) != null) {
			irepo.deleteById(id);
			msg = "Item " + id + " is successfully deleted!";
		}else
```

```
                msg = id + "does not exist!";
            return msg;
        }

        //specific exception
        @ExceptionHandler
        public ResponseEntity<NameErrorResponse>
    handleException(NameNotFoundException ex){

            //create a student error response  obj
            NameErrorResponse error = new NameErrorResponse();

            //set values
            error.setStatus(HttpStatus.NOT_FOUND.value());
            error.setMessage(ex.getMessage());
            error.setTimestamp(System.currentTimeMillis());

            //return response entity
            return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
        }

        //generic exception
        @ExceptionHandler
        public ResponseEntity<NameErrorResponse> handleException(Exception
    ex){

                //create a student error response  obj
            NameErrorResponse error = new NameErrorResponse();

        //set values
            error.setStatus(HttpStatus.BAD_REQUEST.value());
            error.setMessage(ex.getMessage());
            error.setTimestamp(System.currentTimeMillis());

            //return response entity
            return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
        }

    }
```

5.) In the **controller** package, create a class named **ItemController** with the following lines of code:

```
    @RestController
```

```
@RequestMapping(method=RequestMethod.GET, path="/item/api")
#crossorigin is needed only when you're ready to use this backend api in #your frontend app
@CrossOrigin(origins = "http://localhost:3000")
public class ItemController {
	@Autowired
	ItemService iserv;

	public ItemController() {
		// TODO Auto-generated constructor stub
	}

	@GetMapping("/printAuthor")
	public String printAuthor() {
		return "Leah V. Barbaso";
	}

	//C - Create
	@PostMapping("/postNewItem")
	public ItemEntity postItem(@RequestBody ItemEntity Item) {
		return iserv.postItem(Item);
	}

	//R - Read
	@GetMapping("/getAllItems")
	public List<ItemEntity> getAllItems(){
		return iserv.getAllItems();
	}

	//U - Update
	@PutMapping("/putItem")
	public ItemEntity putItem(@RequestParam int id, @RequestBody
ItemEntity newItemDetails) {
		return iserv.putItem(id, newItemDetails);
	}

	//D - Delete
	@DeleteMapping("/deleteItem/{id}")
	public String deleteItem(@PathVariable int id) {
		return iserv.deleteItem(id);
	}

}
```

ENDPOINTS for testing:

printAuthor	http://localhost:8080//item/api/ printAuthor
Insert Record	http://localhost:8080//item/api/ postNewItem

Search All Records	http://localhost:8080//item/api/ getAllItems
Update Record	http://localhost:8080//item/api/ putItem
Delete Record	http://localhost:8080//item/api/ deleteItem/{id}